Sliceable Component

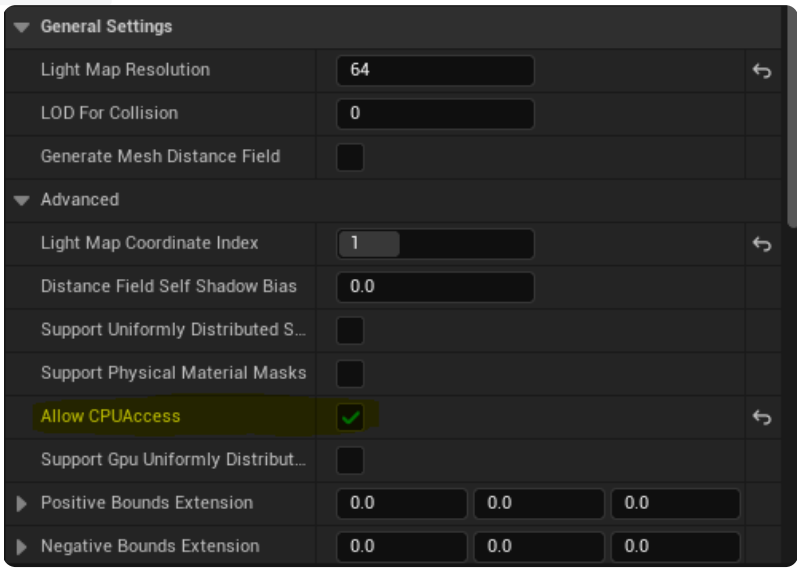
`BP_SliceableComponent` should be applied to any Grabbable that requires the ability to be sliced. It functions by hiding the Grabbable's Static Mesh root component and replacing it, from a rendering standpoint, with a procedural mesh component that can then be sliced along a specified plane position and normal.

Class Details

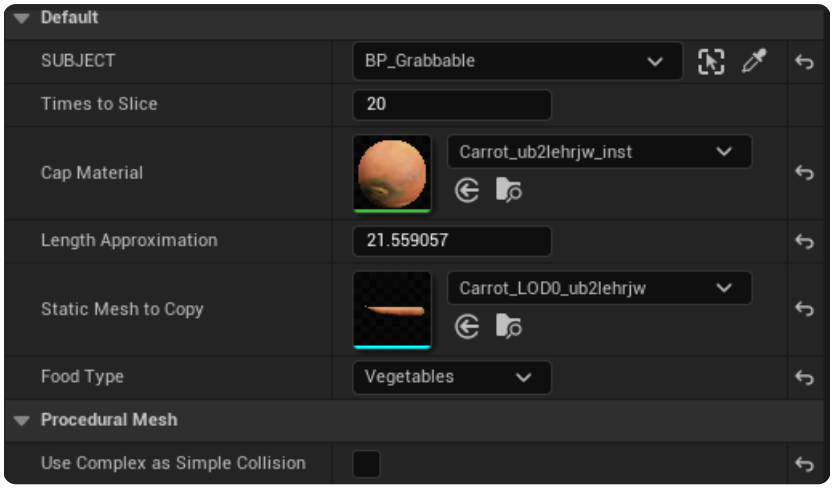
- **Base Class**: Procedural Mesh Component
- **Attachment Protocol**: Root Component of `BP_Grabbable` with zero relative offset
- **Owner**: `BP_Grabbable` to which it is attached

Exposed Properties

- **SUBJECT**: Type `BP_Grabbable`. Used to perform *quasi*-constructor operations at compile-time.
- **TimesToSlice**: Type `int`. Used to limit the number of times that a mesh can be sliced.
- **CapMaterial**: Type `MaterialInterface`. Is applied to the insides of each mesh slice.
- **StaticMeshToCopy:** Type `StaticMesh`. Used in the creation of the procedural mesh. Must have `Allow CPUAccess` setting turned on in its Asset Editor.
- **InProcMesh** Type `ProceduralMeshComponent`. Used in the creation of the procedural mesh *if* `StaticMeshToCopy` is null.



- **FoodType**: Type `ENUMFoodCode`. Used for slicing objective logic.



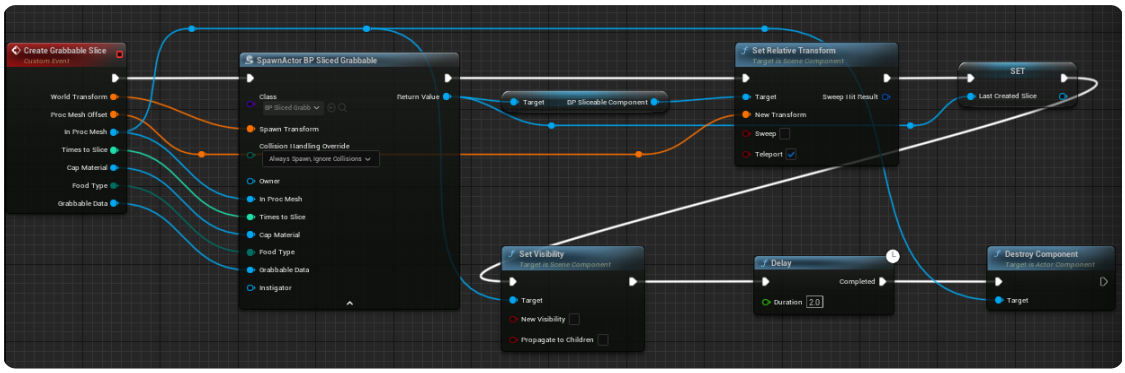
Internal Properties

- **TimesSliced**: Type `int`. Used as a tally for the number of times that a mesh has been sliced.
- **DelayBetweenSlices:** Type `float`. Prevents a mesh from being cut repeatedly in a short time period.
- **IsSliceable:** Type `bool`. Self-explanatory.
- **WasCutBy:** Type `ENUMFoodCode`. Used to evaluate whether or not the correct knife was used to cut a mesh.
- **WasCutOn:** Type `ENUMFoodCode`. Used to evaluate whether or not the mesh was cut on the correct food surface.
- **ContactingSurface:** Type `ENUMFoodCode`. Dynamically updated whenever the mesh collides with a new cutting board.
- **InitialLengthApproximation**: Type `float`. Calculated upon initial procedural mesh creation, and never updated again.
- **PreviousLengthApproximation:** Type `float`. Initialized with value of `InitialLengthApproximation` and subsequently updated with each `Slice` function call.
- **LengthApproximation**: Type `float`. Initialized with value of `InitialLengthApproximation` and subsequently updated with each `GetLength` function call.

Functions

- **BuildMeshFromStaticMeshData**
  - Called from `BP_Grabbable` Construction Script.
  - Checks if `StaticMeshToCopy` is valid:
    - If **true**, then the procedural mesh can be copied from a StaticMeshComponent populated with `StaticMeshToCopy`. Assigns `StaticMeshToCopy` to the Static Mesh property of `SUBJECT`'s `BackupMesh` component, copies a procedural mesh component from `BackupMesh,` sequentially creates complex collision for each section of the procedural mesh, and calls `GetLength` on itself to determine `InitialLengthApproximation`. `LengthApproximation` and `PreviousLengthApproximation` are then assigned the value of `InitialLengthApproximation`.

    - If **false**, then the procedural mesh needs to be created section-by-section using another procedural mesh as a reference. Checks validity of `InProcMesh`; if true, copies materials and mesh sections of `InProcMesh` over to itself via for loops, enables visibility on each section, and adds convex collision to each section. Finally calls `GetLength` on itself to determine `InitialLengthApproximation`. `LengthApproximation` and `PreviousLengthApproximation` are then assigned the value of `InitialLengthApproximation`.
  - Disables line-trace collision response and hides visibility on `SUBJECT`'s `StaticMesh` (ensures that only the procedural mesh component will be visible and generate line-trace events when hit by the knife).
  - Updates collision response settings, enables both query and physics collisions, disables hidden-in-game setting, and disables physics simulation on itself.
- **GetLength**
  - Calculates its bounding sphere radius via the `ComponentBounds` struct, multiplies it by 2 to obtain the diameter, and updates `LengthApproximation` with this new diameter.
  - Returns new `LengthApproximation` value as its output.
- **Slice**
  - Called by `Slice` wrapper function in `BP_Grabbable` on the condition that said `BP_Grabbable` contains a `BP_SliceableComponent`.
  - Receives `PlanePosition` and `PlaneNormal` vectors, `Cutter` `BP_Knife`, and `ToSlice` `ProceduralMeshComponent` as inputs.
  - Checks if `TimesSliced` is equal to zero, which indicates that the object has not been sliced before. If true, adds `Sliced` Gameplay Tag to the object.
  - Updates `IsSliceable` by comparing `TimesToSlice` with `TimesSliced`: if less than, will be true; if greater than or equal to, will be false. Calls `DisableInvisibleMesh` and slices the mesh at the position dictated by `PlanePosition` and angle dictated by `PlaneNormal` if the condition is valid. `OtherHalf` Procedural Mesh is given physics simulation without gravity and used as the basis to instantiate a new `BP_SliceableGrabbable`.
    - The full `BP_SliceableGrabbable`, which is essentially just a `BP_Grabbable` hard-coded with a `BP_SliceableComponent`, takes the newly created `OtherHalf` Procedural Mesh as the "value" of said component. It is spawned into the level by way of a `LineCookGameInstance` function called `CreateGrabbableSlice`.



- `FoodType` of `Cutter` is assigned to `WasCutBy`; `ContactingSurface` is assigned to

`WasCutOn`.
    - `TimesSliced` is incremented.
- The `OnObjectSliced` Event Dispatcher of `SUBJECT` is called, `IsSliceable` is set to false, and `ResizeInvisibleMesh` is called in order to update the slice's collision approximation.
- `RemoveOffsetFromCollision` is called, and then following a `DelayBetweenSlices` delay, `EnableSlicingAgain` is called.
- **DisableInvisibleMesh**
    - Called by `Slice` prior to the actual procedural mesh slicing operation.
    - Disables physics on the Static Mesh root component of `SUBJECT`, ensuring that no physics-related complications will occur when the new procedural mesh component is created.
- **ReenableInvisibleMesh**
    - Called by `Slice` `DelayBetweenSlices` seconds after the slice occurs.
    - Re-enables physics on the Static Mesh root component of `SUBJECT`.
- **EnableSlicingAgain**
    - Re-enables slicing by setting `IsSliceable` to true.
- **ResizeInvisibleMesh**
    - Called by `Slice` following instantiation of the new slice.
    - Call `GetLength` and divide it by `InitialLengthApproximation` to determine the relative size of the object's new procedural mesh (slice).
        - If less than 0.125, replace `SUBJECT`'s `StaticMesh` with the one-eighth carrot mesh.
        - Else, if less than 0.25, replace `SUBJECT`'s `StaticMesh` with the one-quarter carrot mesh.
        - Else, if less than 0.375, replace `SUBJECT`'s `StaticMesh` with the three-eighths carrot mesh.
        - Else, if less than 0.5, replace `SUBJECT`'s `StaticMesh` with the one-half carrot mesh.
        - Else, if less than 0.625, replace `SUBJECT`'s `StaticMesh` with the five-eighths carrot mesh.
        - Else, if less than 0.75, replace `SUBJECT`'s `StaticMesh` with the three-quarters carrot mesh.
        - Else, replace `SUBJECT`'s `StaticMesh` with the seven-eighths carrot mesh.
    - Call the `UpdateInitialMesh` function on `SUBJECT`, which ensures that this mesh will not reset to the full-sized carrot if the object reaches an invalid grabbable zone.
- **RemoveOffsetFromCollision**
    - Called by `Slice` following reactivation of the invisible `StaticMesh` physics.
    - Takes `LastCreatedSlice` `BP_SliceableGrabbable`, `KnifeAngle` `float`, and `PreSliceObjectAngle` `float` as inputs.
    - Contains `OtherHalfLength`, `NewSumOfLengths`, `LengthUncertainty`, `DistanceToMoveThisHalf`, and `DistanceToMoveOtherHalf` as local variables (all of type `float`).
    - First, calls `GetLength` on the `BP_SliceableComponent` of `LastCreatedSlice`. This is then assigned to `OtherHalfLength`.
    - `OtherHalfLength` and `GetLength` of this `BP_SliceableComponent` are added and

assigned to `NewSumOfLengths`.

- Since each bounding sphere is an overestimation, and one additional bounding sphere needs to be taken into account for each slice, there will always be uncertainty within the sum of these lengths. `LengthUncertainty` is calculated by subtracting `PreviousLengthApproximation` (of this component, not `LastCreatedSlice`) from `NewSumOfLengths`.
- `DistanceToMoveThisHalf` is assigned the value of half of the difference between `OtherHalfLength` and half of `LengthUncertainty`. `DistanceToMoveOtherHalf` is assigned the value of half of the difference between the difference between `NewSumOfLengths` and `OtherHalfLength` and half of `LengthUncertainty`. See equations below:

$$\text{DistanceToMoveThisHalf} = \frac{OtherHalfLength - \frac{LengthUncertainty}{2}}{2}$$

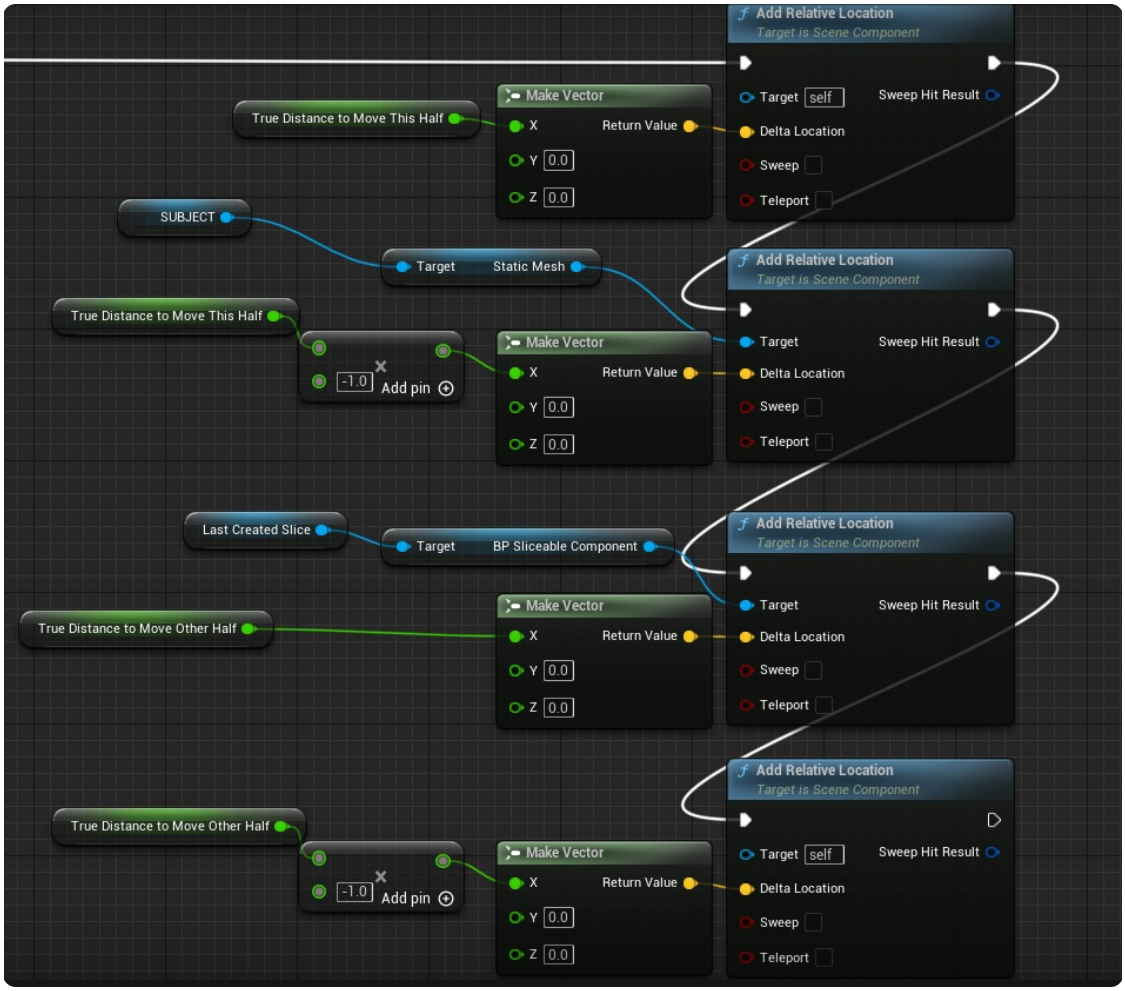
$$\text{DistanceToMoveOtherHalf} = \frac{(NewSumOfLengths - OtherHalfLength) - \frac{LengthUncertainty}{2}}{2}$$

- Before the slices can be moved by these calculated distances, the system needs to determine which slice is ahead of the slicing plane and which is behind it. This is where `KnifeAngle` and `PreSliceObjectAngle` become important, as the slicing plane of the knife will be parallel or inversely parallel to the sliceable object if the models are perpendicular. Thus, `DistanceToMoveThisHalf` is multiplied by the cosine value of the difference between these two angles (in degrees), and `DistanceToMoveOtherHalf` is multiplied by the negation of this value:

$$TrueDistanceToMoveThisHalf = DistanceToMoveThisHalf \times \cos\left(\frac{PreSliceObjectAngle}{KnifeAngle}\right)$$

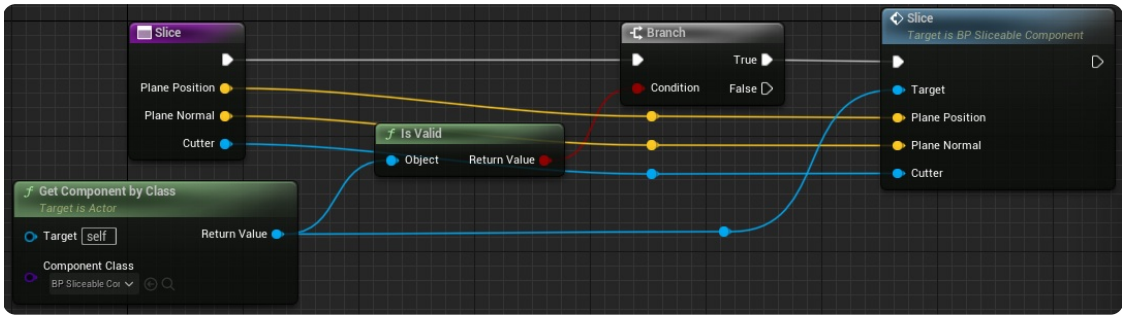
$$TrueDistanceToMoveOtherHalf = DistanceToMoveOtherHalf \times (-1)\cos\left(\frac{PreSliceObjectAngle}{KnifeAngle}\right)$$

- The procedural mesh components are moved by their respective outright true distances (`this` by `TrueDistanceToMoveThisHalf` and `LastCreatedSlice` by `TrueDistanceToMoveOtherHalf )` along their relative X axes, while their respective `StaticMesh` parents are moved by the same amount in the opposite direction. This ensures that the visible procedural mesh model returns to the same spot in which it was instantiated. Lastly, `GetLength` is called.



Dependencies & Details

- `BP_Grabbable` knows whether or not it contains a `BP_SliceableComponent`:



- `BP_SliceableComponent` knows that it is attached to a `BP_Grabbable`.
- `BP_SliceableComponent` knows that `BP_SlicedGrabbable` exists, and that instances of this class contain instances of itself.
- `BP_SliceableComponent` is aware that its game instance is of type `LineCookGameInstance`.
- All default physics and collision properties are handled in `BuildMeshFromStaticMeshData` automatically. There is no need to edit any of these properties in the Details panel (nor is it actually possible to do so).
- **In the `BP_SliceableComponent`'s Procedural Mesh settings, `Use Complex As Simple Collision` MUST be disabled.**